

TCP 3-Way Handshake

Interactive Physical Role-Play: Client, Router, Server & Packet Transmission

6 Team Performers

Client, Router, Server & 3 Packets

Physical Packet Flow

Client <-> Router <-> Server Traversal

3-Step Handshake

SYN -> SYN-ACK -> ACK Synchronization

Introduction & Role-Play Activity Concept

Why a Physical Role-Play?

Visualizing Protocol Mechanics

- **Hop-by-Hop Visibility:** Shows packets traversing intermediate routers rather than instant abstract delivery.
- **Host State Tracking:** Demonstrates live state sign changes (CLOSED -> LISTEN -> ESTABLISHED).
- **Interactive Learning:** Connects theoretical socket code to physical packet movement.

Key Protocol Parameters

Control Flags & Sequence Math

- **SYN (Synchronize):** Requests starting sequence synchronization from receiver.
- **ACK (Acknowledge):** Confirms receipt of sequence bytes ($Ack = Seq + 1$).
- **ISN (Initial Seq No):** Random 32-bit sequence offset generated by Client (x) and Server (y).

Role-Play Execution Rules

Classroom Floor Instructions

- **Packet Performers:** Physically carry flag cards and sequence numbers across the classroom floor.
- **Router Performer:** Inspects destination IP/Port and routes packets between hosts.
- **Host Performers:** Update state signs immediately upon receiving valid packets.

💡 **Fundamental TCP Rule:** Application data (e.g. HTTP requests) cannot flow until the 3-Way Handshake successfully transitions both Client and Server to the ESTABLISHED state.

Our Network Role-Play: 6 Team Roles

Role 1: Client / Sender HOST A (INITIATING HOST)

- **Function:** Initiates connection via socket connect().
- **State Signs:** CLOSED -> SYN-SENT -> ESTABLISHED.
- **Sequence:** Generates Initial Seq Number (ISN_c = x).

Role 2: SYN Packet STEP 1 PACKET PERFORMER

- **Function:** Carries initial connection request.
- **Flags & Parameters:** Flags: [SYN=1, ACK=0], Seq = x, Ack = 0.
- **Path:** Walks from Client -> Router -> Server.

Role 3: Router / Switch INTERMEDIATE NETWORK HOP

- **Function:** Receives packets on ingress port & forwards to destination.
- **Inspection:** Verifies packet headers & routes across network.
- **Realism:** Demonstrates hop-by-hop network traversal.

Role 4: Server / Receiver HOST B (LISTENING HOST)

- **Function:** Listens on port 443 & processes connection requests.
- **State Signs:** LISTEN -> SYN-RCVD -> ESTABLISHED.
- **Sequence:** Generates Server ISN (ISN_s = y).

Role 5: SYN-ACK Packet STEP 2 PACKET PERFORMER

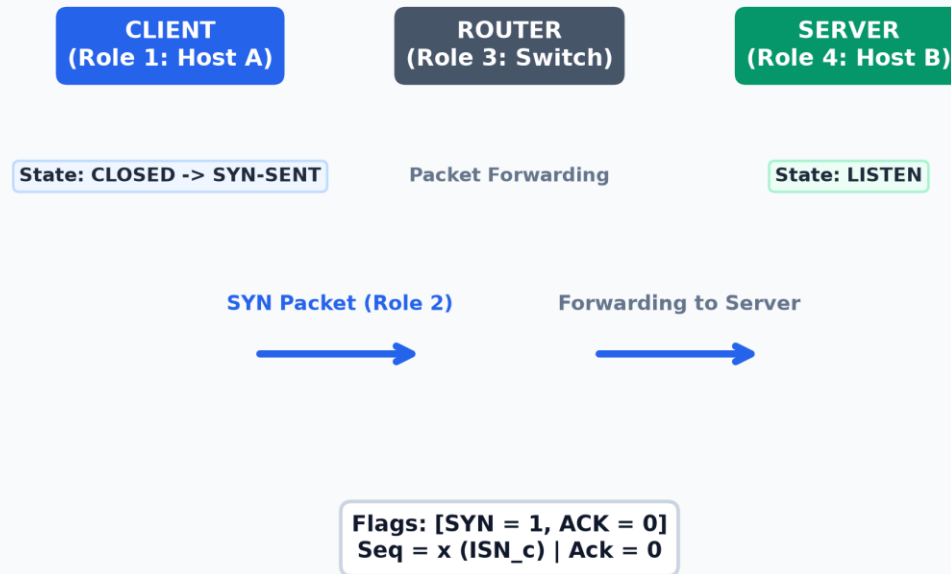
- **Function:** Carries Server's response & sequence proposal.
- **Flags & Parameters:** Flags: [SYN=1, ACK=1], Seq = y, Ack = x+1.
- **Path:** Walks from Server -> Router -> Client.

Role 6: ACK Packet STEP 3 PACKET PERFORMER

- **Function:** Carries final confirmation to establish session.
- **Flags & Parameters:** Flags: [SYN=0, ACK=1], Seq = x+1, Ack = y+1.
- **Path:** Walks from Client -> Router -> Server.

Step 1: Client Connection Request (SYN Exchange)

Step 1: SYN Packet Exchange

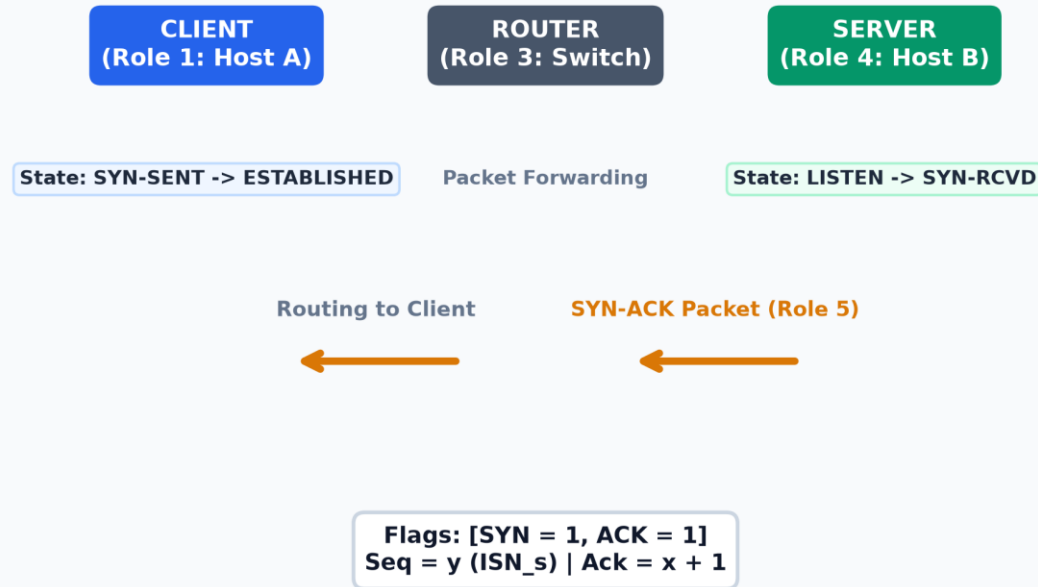


Physical Performer Actions (Step 1)

- **Client (Role 1) [Initiates Connection]**: Calls connect(), generates ISN_c = x (e.g., 1000). Updates state sign: CLOSED -> SYN-SENT.
- **SYN Packet (Role 2) [Dispatches Packet]**: Holds card: [SYN=1, ACK=0], Seq = x, Ack = 0. Walks from Client to Router.
- **Router (Role 3) [Forwards Packet]**: Inspects SYN packet on ingress interface and forwards across classroom to Server.
- **Server (Role 4) [Receives SYN]**: In LISTEN state. Receives SYN, allocates half-open TCB buffer entry in SYN queue.
- **Sequence Rule [Consumes 1 Seq Count]**: SYN flag consumes 1 sequence number (x). Next expected byte will be x + 1.

Step 2: Server Response & Synchronization (SYN-ACK)

Step 2: SYN-ACK Packet Exchange

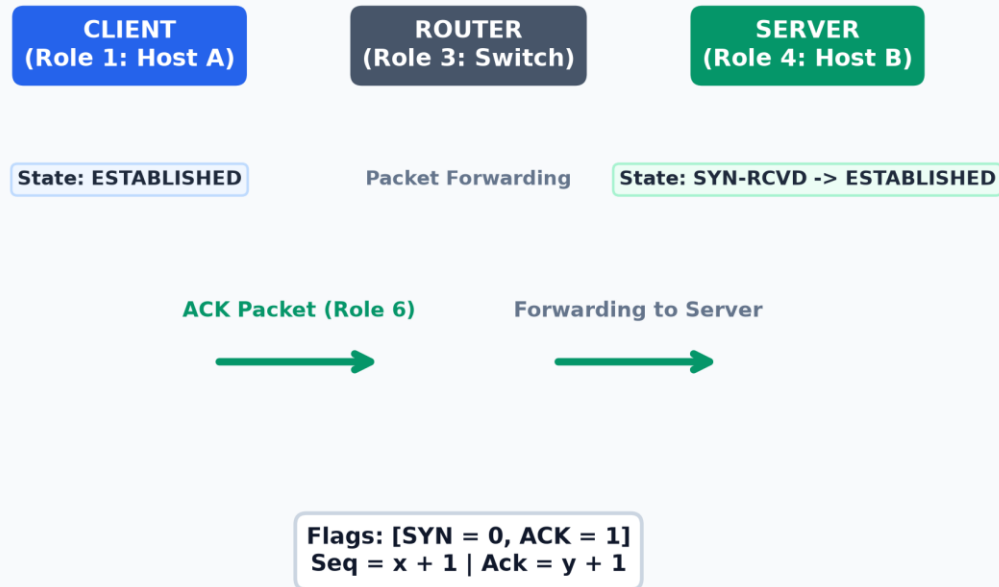


Physical Performer Actions (Step 2)

- **Server (Role 4) [Responds & Syncs]:** Generates Server ISN_s = y (e.g., 5000) and sets Ack = x + 1. Updates state sign: LISTEN -> SYN-RCVD.
- **SYN-ACK Packet (Role 5) [Dispatches Packet]:** Holds card: [SYN=1, ACK=1], Seq = y, Ack = x + 1. Walks from Server to Router.
- **Router (Role 3) [Routes Back]:** Receives SYN-ACK from Server interface and routes back across classroom to Client.
- **Client (Role 1) [Receives SYN-ACK]:** Verifies Ack = x + 1. Immediately updates state sign: SYN-SENT -> ESTABLISHED!
- **Milestone [Client Connected]:** Client host is now ready to transmit application payload after final ACK.

Step 3: Client Confirms Connection (ACK Exchange)

Step 3: ACK Packet Exchange



Physical Performer Actions (Step 3)

- **Client (Role 1) [Sends Final ACK]:** Sets Seq = x + 1 and Ack = y + 1. Clears SYN flag (SYN=0).
- **ACK Packet (Role 6) [Dispatches Packet]:** Holds card: [SYN=0, ACK=1], Seq = x + 1, Ack = y + 1. Walks from Client to Router.
- **Router (Role 3) [Forwards ACK]:** Receives final ACK segment and delivers it to Server interface.
- **Server (Role 4) [Completes Handshake]:** Receives ACK, verifies Ack = y + 1. Updates state sign: SYN-RCVD -> ESTABLISHED!
- **Outcome [Bi-Directional Sync]:** Both sockets are now ESTABLISHED. Application data transfer is unlocked!

Connection Established & Role-Play Activity Summary

1. Physical Packet Traversal

CLIENT <-> ROUTER <-> SERVER

- **Router Mediation:** Packets physically traveled through intermediate router hop (Role 3).
- **Flag Discipline:** SYN requested sync; SYN-ACK acknowledged & proposed; ACK confirmed.
- **Latency:** Complete 3-step physical exchange took 1 Round Trip Time (1 RTT).

2. Sequence & Ack Number Math

BYTE OFFSET AGREEMENT

- **Client Offset:** Client $ISN_c = x$ -> Server $ACK = x + 1$ -> Next Client Seq = $x + 1$.
- **Server Offset:** Server $ISN_s = y$ -> Client $ACK = y + 1$ -> Next Server Seq = $y + 1$.
- **Sequence Cost:** SYN control flag consumed 1 sequence count without carrying data payload.

3. Host State Machine Alignment

SYNCHRONIZED STATE SIGNS

- **Client Path:** CLOSED -> SYN-SENT -> ESTABLISHED (Active Open).
- **Server Path:** CLOSED -> LISTEN -> SYN-RCVD -> ESTABLISHED (Passive Open).
- **Final State:** Both Client and Server hold 'ESTABLISHED' signs visible to classroom.

4. Transition to Application Data

PAYLOAD TRANSMISSION UNLOCKED

- **Session Active:** Full-duplex reliable connection is now active between Host A and Host B.
- **Application Flow:** Client can now transmit HTTP GET request over open TCP socket.
- **Role-Play Wrap:** Applause for our 6 Role-Play Performers and Narrator!

 **Role-Play Activity Conclusion:** Our 6 role performers demonstrated how TCP establishes a reliable connection across a network router in 3 physical steps. By synchronizing sequence numbers x and y , both Client and Server reached the ESTABLISHED state. Great job team!